

Before we write **actual triggers**, it's crucial to **list all possible trigger instances** for your database. Triggers are generally used to:

- Automatically update related tables
- Maintain history/audit tables
- Enforce consistency or calculations
- Restrict invalid operations

I'll go **table by table** in order, starting with the first table, and list all **possible trigger events**. Later, we can implement them one by one.

1 categories table

Trigger Instances:

1. **AFTER INSERT** → Log creation of a new category in a `categories_history` table (if you want audit).
2. **AFTER UPDATE** → Log any updates (brand, model, type, screen_size, color).
3. **BEFORE DELETE** → Prevent deletion if products exist under this category.
4. **AFTER DELETE** → Optionally log deletion in `categories_history`.

2 suppliers table

Trigger Instances:

1. **AFTER INSERT** → Add to `suppliers_history` for tracking new suppliers.
2. **AFTER UPDATE** → Log changes to supplier details.
3. **BEFORE DELETE** → Prevent deletion if products exist from this supplier.
4. **AFTER DELETE** → Log deletion in `suppliers_history`.

3 products table

Trigger Instances:

1. **AFTER INSERT** →
 - Update `inventory` (initial stock 0 or based on first purchase).
 - Insert into `products_history` with `change_type = 'add'`.
2. **AFTER UPDATE** →
 - Insert into `products_history` with `change_type = 'edit'`.
 - Update `inventory` `unit_cost` or derived values if `cost_price` changed.
3. **BEFORE DELETE** → Prevent deletion if orders, stocks, or sales exist.
4. **AFTER DELETE** → Insert into `products_history` with `change_type = 'delete'`.

4 staff table

Trigger Instances:

1. **AFTER INSERT** → Log in `staff_history` if maintained.
2. **AFTER UPDATE** →
 - Track role change.
 - Track location change.
3. **BEFORE DELETE** → Prevent deletion if staff is referenced in orders, invoices, stocks, etc.
4. **AFTER DELETE** → Optionally log in `staff_history`.

5 products_history table

- Usually no triggers here (it's already a history table).

6 orders table

Trigger Instances:

1. **AFTER INSERT** →
 - Automatically generate `order_items` if pre-populated.
 - Update `inventory` (subtract reserved quantities).
 - Optionally log in `orders_history`.
2. **AFTER UPDATE** →
 - Update `orders_history` with `change_type = edit`.
 - Update `inventory` if order status changes (e.g., returned).
3. **BEFORE DELETE** → Prevent deletion if sales or invoice already created.
4. **AFTER DELETE** → Log deletion in `orders_history`.

7 order_items table

Trigger Instances:

1. **AFTER INSERT** →
 - Update `stocks` and `inventory` to deduct sold quantity.
 - Insert into `order_items_history` if needed.
2. **AFTER UPDATE** → Adjust inventory and stocks if `qty` or `unit_price` changed.
3. **BEFORE DELETE** → Prevent deletion if sales or returns exist.
4. **AFTER DELETE** → Adjust inventory/stock and log history.

8 orders_history table

- Already a history table; triggers mostly happen on `orders`.

9 returns table

Trigger Instances:

1. **AFTER INSERT** →
 - Update `stocks` and `inventory` to add returned quantity.
 - Adjust `sales` table (negative amount).
 - Update `orders_history` to mark returned status.
2. **AFTER UPDATE** → Track changes in `returns_history` if maintained.
3. **BEFORE DELETE** → Prevent deletion if linked to `return_items` or `sales`.
4. **AFTER DELETE** → Update inventory and log history.

10 return_items table

Trigger Instances:

1. **AFTER INSERT** →
 - Add returned quantity to `stocks` and `inventory`.
 - Update `sales` with negative `amount`.
2. **AFTER UPDATE** → Adjust `stocks` and `inventory` if `qty_returned` changes.
3. **BEFORE DELETE** → Prevent deletion if linked to invoices.
4. **AFTER DELETE** → Adjust `stocks` and `inventory`.

1 1 locations table

Trigger Instances:

1. **AFTER INSERT** → Initialize `stocks` / `inventory` or staff assignments if needed.
2. **AFTER UPDATE** → Track changes in `location_status` (active/inactive/closed).
3. **BEFORE DELETE** → Prevent deletion if linked with staff, orders, stocks, or inventory.

1 2 stocks table

Trigger Instances:

1. **AFTER INSERT** → Update `inventory` total quantity for the product.
2. **AFTER UPDATE** → Adjust `inventory` if `qty_stocked` changes.
3. **BEFORE DELETE** → Prevent deletion if sales exist.
4. **AFTER DELETE** → Update `inventory` to remove quantity.

1 3 stocks_history table

- Mainly triggered by `stocks` changes. No triggers on this table itself.

1 4 inventory table**Trigger Instances:**

1. **AFTER UPDATE** → Update `inventory_history` for tracking changes.

1 5 inventory_history table

- Triggered by `inventory` changes; no triggers on itself.

1 6 purchases table**Trigger Instances:**

1. **AFTER INSERT** →
 - Add purchased quantity to `stocks` of relevant location.
 - Update `inventory`.
 - Log in `stocks_history` and `inventory_history`.
2. **AFTER UPDATE** → Adjust `stocks` and `inventory` if quantity or unit_cost changes.
3. **BEFORE DELETE** → Prevent deletion if inventory already updated.
4. **AFTER DELETE** → Reverse the stock and inventory adjustments.

1 7 stock_movement_records table**Trigger Instances:**

1. **AFTER INSERT** →
 - Deduct quantity from `from_location` stock.
 - Add quantity to `to_location` stock.
 - Update `inventory`.
2. **AFTER UPDATE** → Adjust stock movement if `quantity_moved` changes.
3. **BEFORE DELETE** → Prevent deletion if stock movement is confirmed.
4. **AFTER DELETE** → Reverse stock changes.

1 8 stock_movement_history table

- Triggered by `stock_movement_records`. No triggers on itself.

1 9 roles table**Trigger Instances:**

1. **BEFORE DELETE** → Prevent deletion if staff exists with this role.

2. **AFTER UPDATE** → Optionally log role name changes in a `roles_history`.

2 0 `invoices` table

Trigger Instances:

1. **AFTER INSERT** →
 - Create related `order` if new customer.
 - Update `stocks` and `inventory` with sold quantities.
 - Insert related `sales` records.
 2. **AFTER UPDATE** → Track edits in `invoice_history` if implemented.
 3. **BEFORE DELETE** → Prevent deletion if linked sales exist.
-

2 1 `invoice_items` table

Trigger Instances:

1. **AFTER INSERT** → Update `stocks` and `inventory`.
 2. **AFTER UPDATE** → Adjust stock if quantity/unit_price changes.
 3. **BEFORE DELETE** → Prevent deletion if linked to sales.
 4. **AFTER DELETE** → Reverse stock and inventory changes.
-

2 2 `sales` table

Trigger Instances:

1. **AFTER INSERT** → Update `inventory` and `stocks` (positive for sales, negative for returns).
 2. **AFTER UPDATE** → Adjust inventory and net_profit_loss if quantity/amount changes.
 3. **AFTER DELETE** → Reverse inventory adjustments.
-

2 3 `sales_history` table

- Triggered by `sales` changes. No triggers on itself.
-

💡 Summary of main trigger types:

- **AFTER INSERT:** Auto-update related tables, create history, update inventory/stocks, generate sales.
- **AFTER UPDATE:** Track changes, recalculate derived fields, update history.
- **BEFORE DELETE:** Prevent deletion if dependencies exist.
- **AFTER DELETE:** Reverse updates and log history.